

Master program
" **Advanced Analytics for Business** "

Advanced Visual Text Analytics– Natural Language Processing



Universitatea Națională de Știință și Tehnologie Politehnica București
Facultatea de Automatică și Calculatoare



Course 6 Word Embeddings



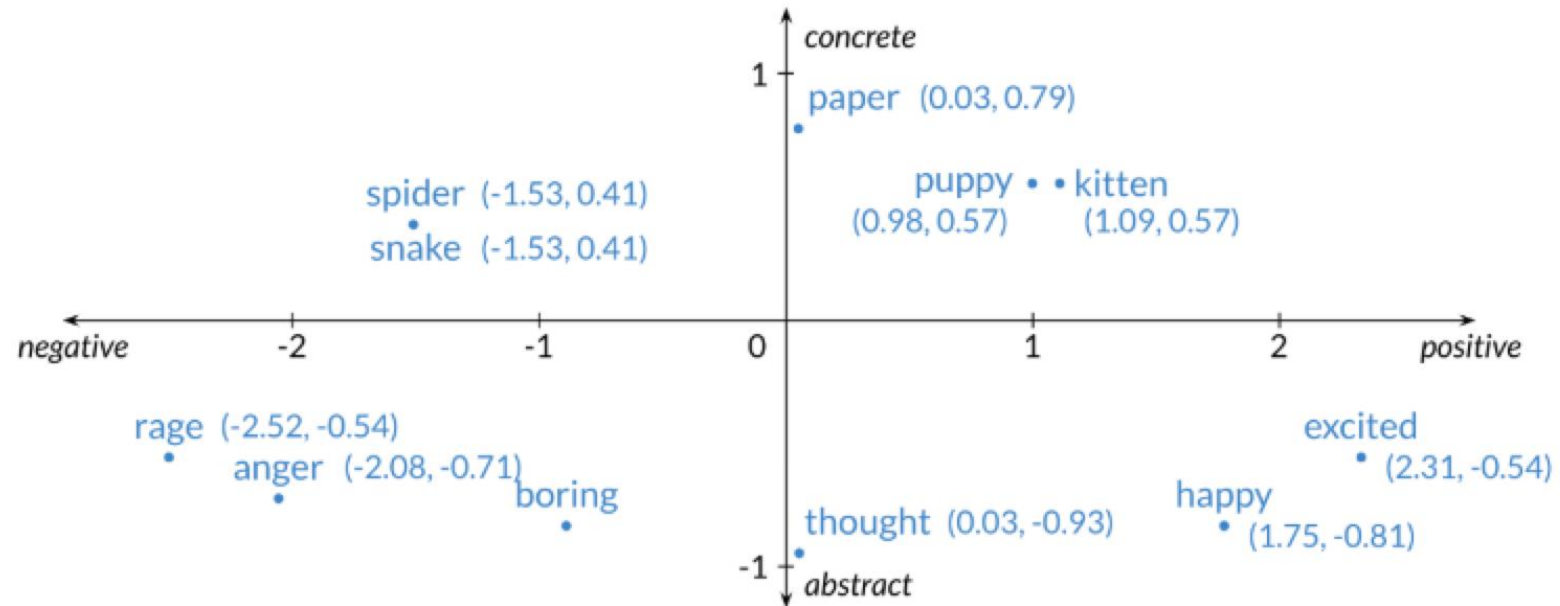
Lecture content

- Word embeddings
 - Definition.
 - How to create word embeddings.
 - Word embeddings methods.
 - Continuous Bag of Words (CBoW) model definition.
 - Cleaning and tokenization.
 - Sliding window of words.
 - Transforming words into vectors.
 - CBoW model architecture.
 - CBoW model dimensions.
 - CBoW model activation functions.
 - CBoW cost function.
 - CBoW training process.
 - Extracting word embeddings vectors.
- Python functions
 - Process data.



- Word embeddings are a type of word representation that allows words with similar meaning to have similar representation.
- Word embeddings are in fact a class of techniques where individual words are represented as real-valued vectors in a predefined vector space.
- One of the benefits of using dense and low-dimensional vectors is computational.
- Basic word representations:
 - Integers.
 - (*pros*: simple, *cons*: little semantic sense).
 - One-hot vectors.
 - (*pros*: simple, *cons*: huge data representation and encode no meaning)
 - Word embeddings.

Word embeddings



Pros:

low dimensions lower than V .

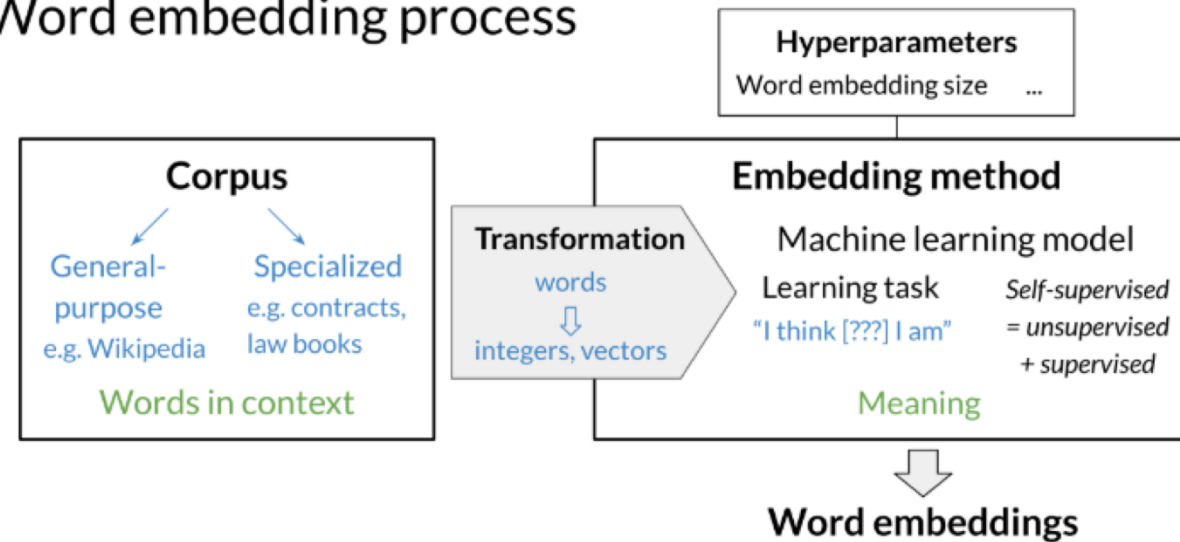
allow to encode meaning.

*DeepLearning.AI

How to create word embeddings

- Inputs:
 - Corpus of text (could be general purpose (Wikipedia), or specialized (law, banking, technology information)).
- Process:
 - *Self-supervised* learning task: corpus is unlabeled (unsupervised), but data itself provides context which will be translated as labeled data (supervised).

Word embedding process



*DeepLearning.AI

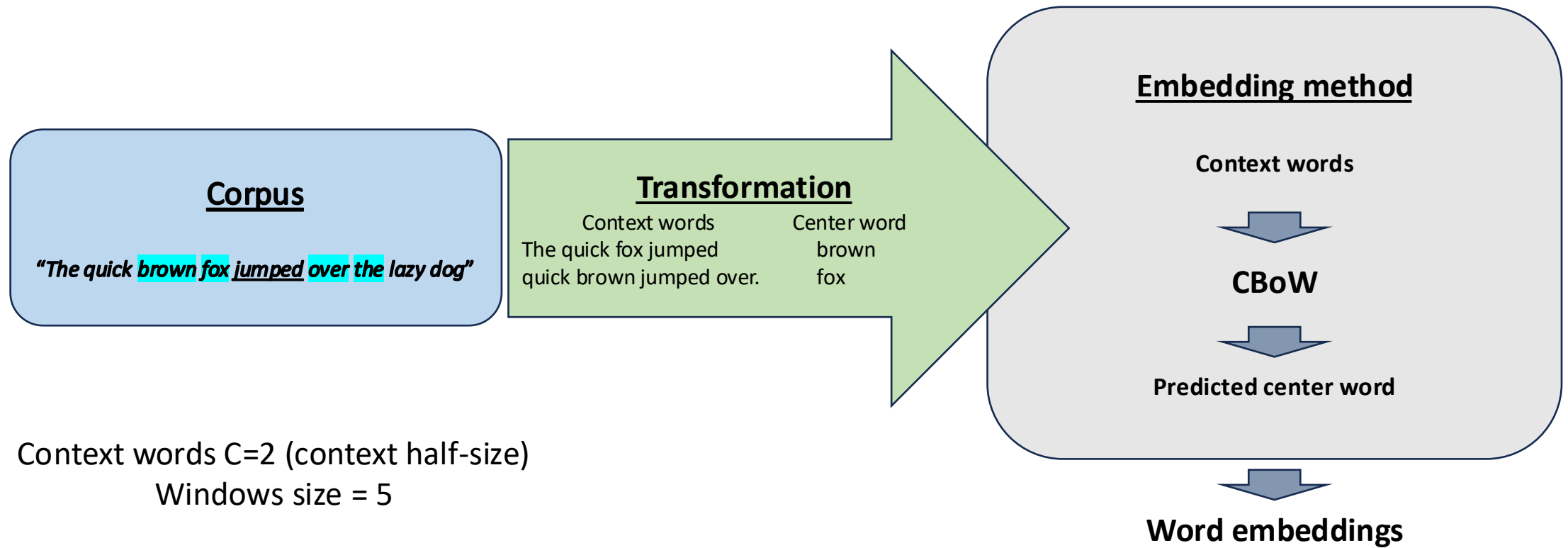


Word embeddings methods

- **Word2Vec** (Google, 2013): stand alone word embedding from a text corpus.
- **Continuous Bag of Words** (CBoW): learns the embeddings based on its context/ neighbors' words.
- **Continuous Skip-Gram** (CSG): learns by predicting the surrounding words given a current word.
- **Global Vectors** (GloVe) (Stanford, 2014): using Latent Semantic Analysis (LSA) represents contextual meaning of words by statistical computations applied to a large corpus of text.
- Deep learning, contextual embeddings – words have different embeddings depending on their context:
- **BERT** (Google, 2018).
- **ELMo** (Allen Institute for AI, 2018).
- **GPT-5** (Open AI, 2025)

Continuous Bag of Words (CBoW) Model

- In the case of CBoW the goal of the task is to predict a missing word based on surrounding/ neighbors' words :



Context words $C=2$ (context half-size)
Windows size = 5

Cleaning and tokenization

- Lower case or upper case: the quick brown fox jumped over the lazy dog.
- Punctuation: , ! . ? → . or remove “ ”
- Numbers can be kept as is
- Special characters could be removed
- Emoji interpretations

```
def clean_and_tokenize(text):  
    # Lowercasing  
    text = text.lower()  
  
    # Replace punctuation with a period (.)  
    text = re.sub(r"[!?!;-]+", ".", text)  
  
    # Tokenization  
    tokens = word_tokenize(text)  
  
    # Remove stopwords  
    stop_words = set(stopwords.words("english"))  
    tokens = [word for word in tokens if word not in stop_words]  
  
    # Lemmatization  
    lemmatizer = WordNetLemmatizer()  
    tokens = [lemmatizer.lemmatize(word) for word in tokens]  
  
    return tokens
```

Sliding window

- Function to implement sliding window when parsing the corpus

Corpus

"The quick brown fox jumped over the lazy dog now"

```
def get_windows(words, C):  
    for i in range(C, len(words) - C): # Sliding over the words  
        context = words[i - C:i] + words[i + 1:i + C + 1] # Selecting context words  
        center = words[i] # Selecting the center word  
        yield context, center # Return (context, center) pair but don't exit the function
```

!!! Check what is the difference between *return* and *yield*

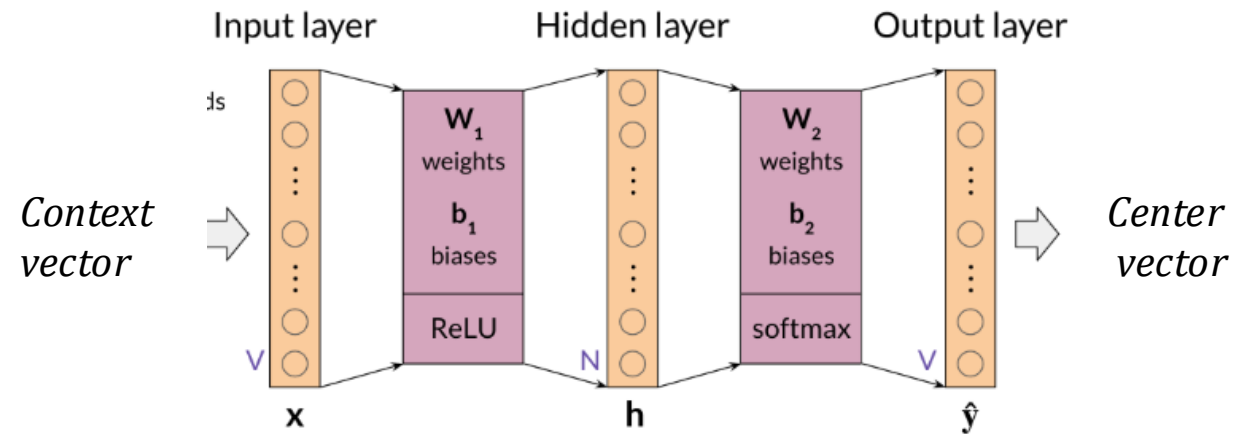
Transforming words into vectors

- Transform context vectors into one single vector:

$$\begin{pmatrix} \text{brown} \\ \text{dog} \\ \text{fox} \\ \text{jumped} \\ \text{lazy} \\ \text{now} \\ \text{over} \\ \text{quick} \\ \text{the} \end{pmatrix}
 \begin{matrix} \text{brown} & \text{fox} & \text{over} & \text{the} \end{matrix}
 \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}
 +
 \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}
 +
 \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}
 +
 \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}
 /4 =
 \begin{pmatrix} 0.25 \\ 0 \\ 0.25 \\ 0 \\ 0 \\ 0 \\ 0.25 \\ 0 \\ 0.25 \end{pmatrix}$$

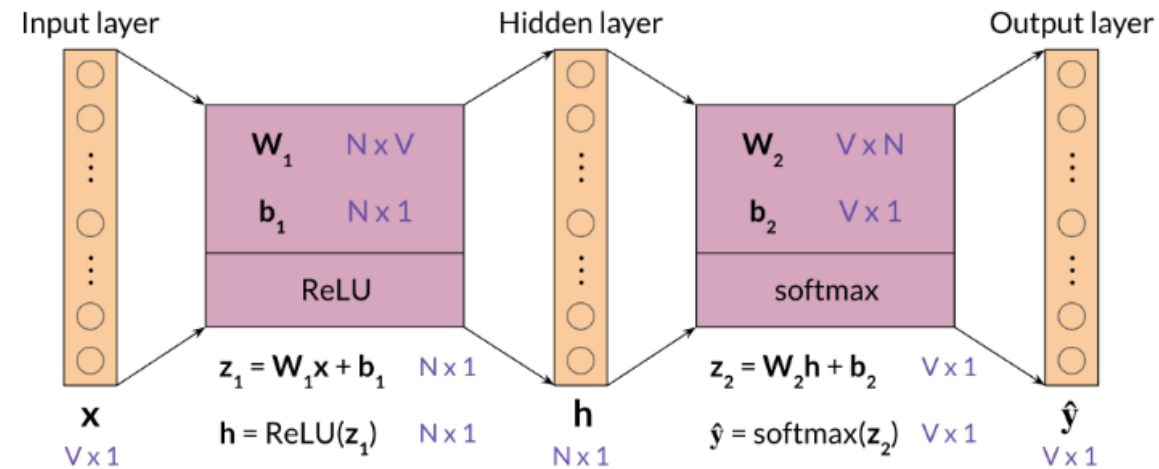
Architecture of CBoW Model

- Context vector is the average of all context vectors ($V=9$)

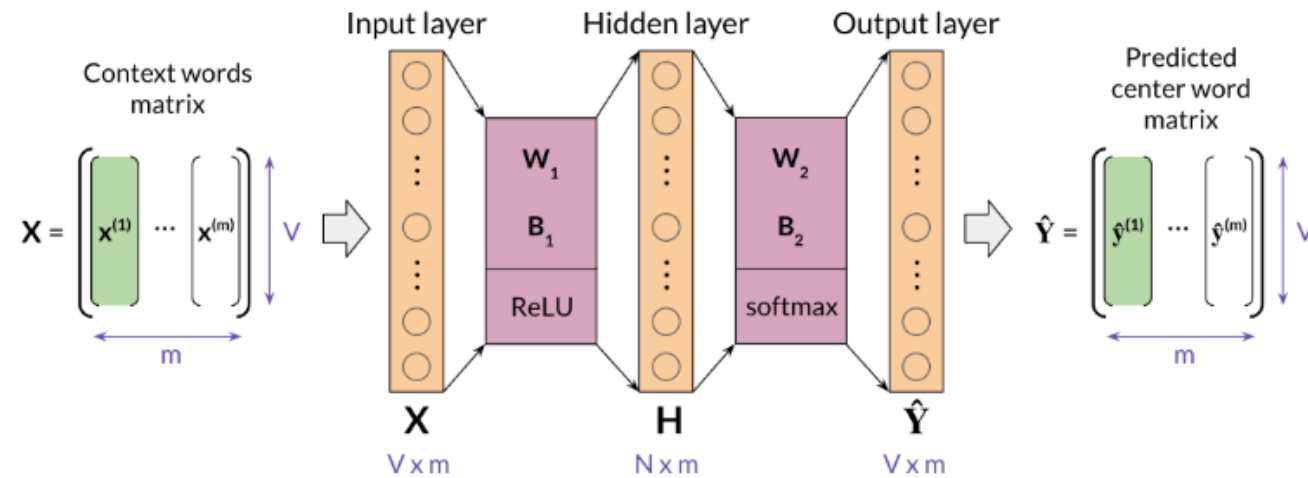


Dimensions for CBoW architecture – one input

- Equations are:
 - $z_1 = W_1x + b_1$
 - $h = \text{ReLU}(z_1)$
 - $z_2 = W_2h + b_2$
 - $\hat{y} = \text{softmax}(z_2)$

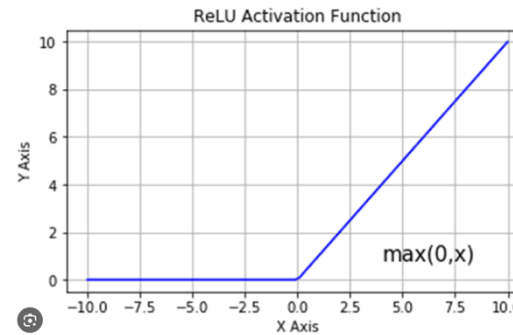


Dimensions for CBoW architecture – batch input



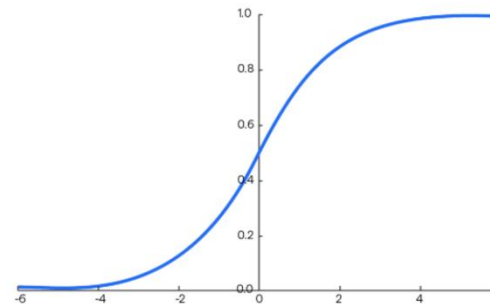
CBoW activation functions

- **ReLU** – Rectified linear unit – function is linear for values greater than zero, meaning it has a lot of the desirable of a linear activation function when training a neural network, but it is a nonlinear function as negative values are always output as zero.



- **Softmax** – a function that converts input values to 0-1 values that sum up to 1.

- $$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$



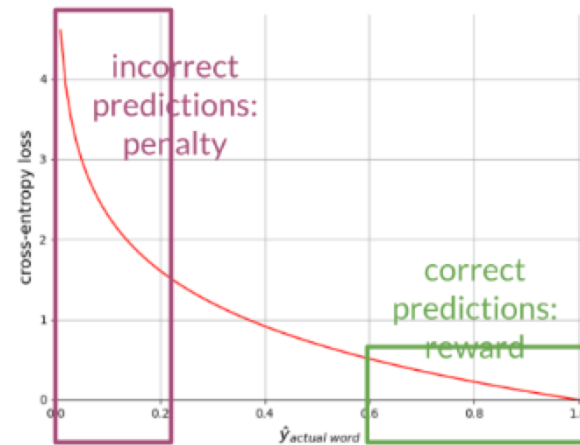
Training CBoW model – Cost function

- Cost function – cross entropy loss for one input:

$$J = - \sum_{k=1}^V y_k \log \hat{y}_k$$

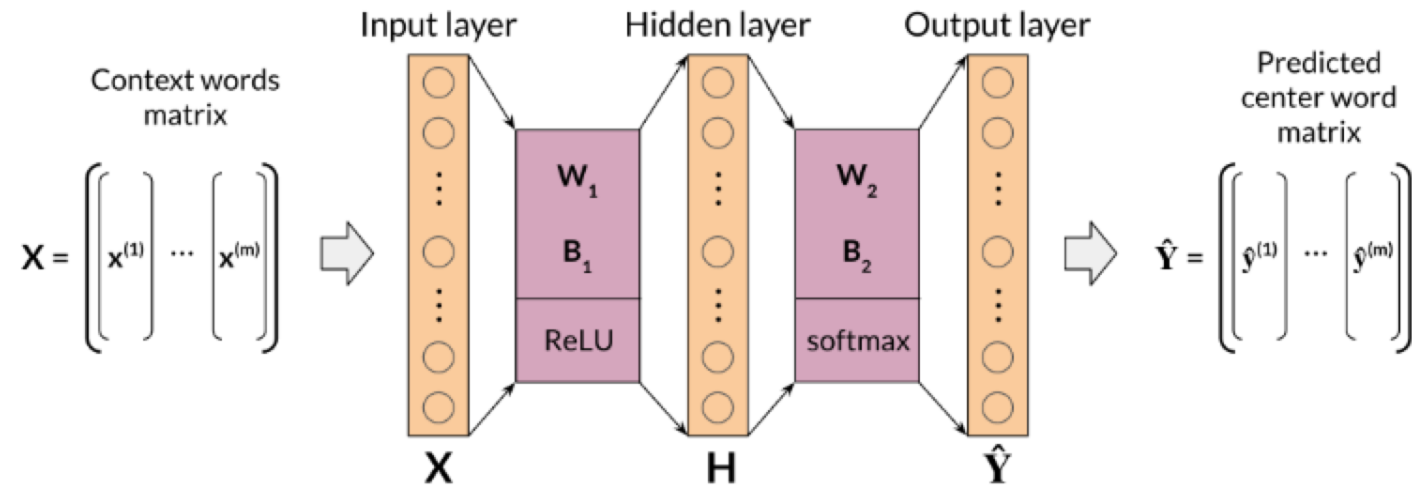
- Cost function for batch:

$$J_{batch} = - \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^V y_j^{(i)} \log \hat{y}_j^{(i)}$$



Training CBoW model – Forward propagation

- Equations are:
 - $Z_1 = W_1X + B_1$
 - $H = \text{ReLU}(Z_1)$
 - $Z_2 = W_2H + B_2$
 - $\hat{Y} = \text{softmax}(Z_2)$



Training CBoW model – Backpropagation and Gradient Descent

- **Backpropagation** – algorithm that use gradient descent algorithms to update network weights. Calculate partial derivatives of cost in relation with weights and biases:

- $\frac{\partial J_{batch}}{\partial W_1}, \frac{\partial J_{batch}}{\partial W_2}, \frac{\partial J_{batch}}{\partial b_1}, \frac{\partial J_{batch}}{\partial b_2}$

- Update de weights and biases – **gradient descent**:

- $W_1 := W_1 - \alpha \frac{\partial J_{batch}}{\partial W_1},$

- $W_2 := W_2 - \alpha \frac{\partial J_{batch}}{\partial W_2},$

- $b_1 := b_1 - \alpha \frac{\partial J_{batch}}{\partial b_1},$

- $b_2 := b_2 - \alpha \frac{\partial J_{batch}}{\partial b_2}$

Python functions

- Load and preprocess data

```
# Load, tokenize and process the data
import re
with open('./data/shakespeare.txt') as f:
    data = f.read()
data = re.sub(r'[!?;-]', '.', data)
data = nltk.word_tokenize(data)
data = [ch.lower() for ch in data if ch.isalpha() or ch == '.']
print("Number of tokens:", len(data), '\n', data[:15])

# Load the Regex-module

# Punctuations are replaced by .
# Tokenize string to words
# Lower case and drop non-alphabetical tokens
# print data sample

# Compute the frequency distribution of the words in the dataset (vocabulary)
fdist = nltk.FreqDist(word for word in data)
print("Size of vocabulary: ", len(fdist) )
print("Most frequent tokens: ", fdist.most_common(20) ) # print the 20 most frequent words and their freq.
```

Python functions

- Get word to index and Index to word from data set:

```
def get_dict(data):  
    """  
    Input:  
        data: from which to extract a list of indices  
    Outputs:  
        word2Ind: returns dictionary mapping the word to its index  
        Ind2word: returns dictionary mapping the index to its word  
    """  
    # converts data into a set & removes duplicate words and convert back to list  
    # sorts the words in alphabetical order  
    words = sorted(list(set(data)))  
    n = len(words) # number of unique words  
    idx = 0  
    word2Ind = {}  
    Ind2word = {}  
    for k in words:  
        word2Ind[k] = idx  
        Ind2word[idx] = k  
        idx += 1  
    return word2Ind, Ind2word  
  
# get word to index and index to word.  
word2Ind, Ind2word = get_dict(data)  
V = len(word2Ind)  
print("Size of vocabulary: ", V)
```

Python functions

- Initialize weights and biases vectors

```
def initialize_model(N,V, random_seed=1):  
    '''  
    Inputs:  
        N: dimension of hidden vector  
        V: dimension of vocabulary  
    Outputs:  
        W1, W2, b1, b2: initialized weights and biases  
    '''  
    #initialise randomly the weights and biases  
    np.random.seed(random_seed)  
    # W1 has shape (N,V)  
    W1 = ..  
  
    # W2 has shape (V,N)  
    W2 = ..  
  
    # b1 has shape (N,1)  
    b1 = ..  
  
    # b2 has shape (V,1)  
    b2 = ..  
  
    return W1, W2, b1, b2
```

Python functions

- Calculate softmax function:

```
def softmax(z):  
    '''  
    Inputs:  
        z: output scores from the hidden layer  
    Outputs:  
        yhat: prediction (estimate of y)  
    '''  
    # Calculate yhat (softmax)  
    yhat = np.exp(z)/np.sum(np.exp(z), axis=0)  
    return yhat
```

Python functions

- Forward propagation:

```
def forward_prop(x, W1, W2, b1, b2):  
    ...  
    Inputs:  
        x: average one hot vector for the context  
        W1, W2, b1, b2: matrices and biases to be learned  
    Outputs:  
        z: output score vector  
    ...  
    # Calculate h  
    h = ..  
  
    # Apply the relu on h,  
    # store the relu in h  
    h = ..  
  
    # Calculate z  
    z = ..  
  
    return z, h
```



Python functions

- Cost function:

```
# compute_cost
def compute_cost(y, yhat, batch_size):

    # cost function
    logprobs = np.multiply(np.log(yhat), y)
    cost = - 1/batch_size * np.sum(logprobs)
    cost = np.squeeze(cost)
    return cost
```

Python functions

- Backpropagation:

```
def back_prop(x, yhat, y, h, W1, W2, b1, b2, batch_size):  
    """  
    Inputs:  
        x: average one hot vector for the context  
        yhat: prediction (estimate of y)  
        y: target vector  
        h: hidden vector  
        W1, W2, b1, b2: matrices and biases  
        batch_size: batch size  
    Outputs:  
        grad_W1, grad_W2, grad_b1, grad_b2: gradients of matrices and biases  
    """  
    # Compute z1 as "W1·x + b1"  
    z1 = np.dot(W1, x) + b1  
    # Compute l1 as W2^T (Yhat - Y)  
    l1 = ..  
  
    # if z1 < 0, then l1 = 0  
    # otherwise l1 = l1  
  
    l1[z1 < 0] = 0 # use "l1" to compute gradients below  
  
    # compute the gradient for W1  
    grad_W1 = ..  
  
    # Compute gradient of W2  
    grad_W2 = ..  
  
    # compute gradient for b1  
    grad_b1 = ..  
  
    # compute gradient for b2  
    grad_b2 = ..  
  
    return grad_W1, grad_W2, grad_b1, grad_b2
```

Python functions

- Gradient descent:

```
def gradient_descent(data, word2Ind, N, V, num_iters, alpha=0.03,
                    random_seed=282, initialize_model=initialize_model,
                    get_batches=get_batches, forward_prop=forward_prop,
                    softmax=softmax, compute_cost=compute_cost,
                    back_prop=back_prop):

    """
    Inputs:
    data: text, word2Ind: words to Indices
    N: dimension of hidden vector ,V: dimension of vocabulary
    num_iters: number of iterations
    random_seed: random seed to initialize the model's matrices and vectors
    get_batches: function to get the data in batches (function in utils.py)
    forward_prop: your implementation of the function to perform forward propagation
    softmax: your implementation of the softmax function
    compute_cost: cost function (Cross entropy)
    back_prop: your implementation of the function to perform backward propagation

    Outputs:
    W1, W2, b1, b2: updated matrices and biases after num_iters iterations
    """
    #W1=(N,V) and W2=(V,N)
    W1, W2, b1, b2 = initialize_model(N,V, random_seed=random_seed)
    batch_size = 128
    iters = 0
    C = 2

    for x, y in get_batches(data, word2Ind, V, C, batch_size):
        # get z and h
        z, h = ..
        # get yhat
        yhat = softmax(z)
        # get cost
        cost = ..
        if (iters+1) % 10 == 0:
            print(f"iters: {iters + 1} cost: {cost:.6f}")
        # get gradients
        grad_W1, grad_W2, grad_b1, grad_b2 = ..
        # update weights and biases
        W1 -= alpha*grad_W1
        W2 -= alpha*grad_W2
        b1 -= alpha*grad_b1
        b2 -= alpha*grad_b2

        iters +=1
        if iters == num_iters:
            break
        if iters % 100 == 0:
            alpha *= 0.66

    return W1, W2, b1, b2
```

Python functions

- Word vectors visualization:

```
from matplotlib import pyplot
%config InlineBackend.figure_format = 'svg'
words = ['king', 'queen', 'lord', 'man', 'woman', 'dog', 'wolf',
         'rich', 'happy', 'great', 'terrible', 'sad']

embs = (W1.T + W2)/2.0

# given a list of words and the embeddings, it returns a matrix with all the embeddings
idx = [word2Ind[word] for word in words]
X = embs[idx, :]
print(X.shape, idx) # X.shape: Number of words of dimension N each

result= compute_pca(X, 2)
pyplot.scatter(result[:, 0], result[:, 1])
for i, word in enumerate(words):
    pyplot.annotate(word, xy=(result[i, 0], result[i, 1]))
pyplot.show()
```



Python functions



- Put all together:

- ***Good luck 😊***